# Obsidian ERP v4.0 — Architecture Document (Part 1 of 4)

# Foundation, Strategic Vision & Core Architecture

> **Product:** Obsidian ERP
> **Company:** VersaLabs Studio
> **Architect:** Kidus Abdula
> **Document Version:** 4.0.0
> **Last Updated:** 2026-05-31
> **Status:** APPROVED — Implementation Ready
> **Supersedes:** Pana ERP v3.0 (ARCHITECTURE_V3.md)

---

## Table of Contents



---

## 1. Strategic Context

### 1.1 Why V4?

Obsidian ERP v4.0 is **not** a rewrite. It is a strategic **increment** on the battle-tested V3 architecture with three mandates:

| Mandate | Rationale |
|---|---|
| **Full UX Overhaul** | V3 pilot in Addis Ababa revealed that premium UI alone doesn't solve usability. V4 shifts from "beautiful forms" to "guided flows" — button-by-button simplicity over form-field mastery. |
| **AI-Native Integration** | Natural language interaction layer that lets users complete ERP workflows via conversation. This is not a chatbot — it's an intelligent automation layer that executes real system operations. |
| **End-to-End Completeness** | Every module from ERPNext must be present, deployed, and fully usable. No "coming soon" — every workflow from Lead to Cash to GL must function on day one. |

### 1.2 What V3 Taught Us

The V3 pilot with SMEs in Addis Ababa, Ethiopia revealed critical insights:

| Observation | V3 Reality | V4 Response |
|---|---|---|
| **Form Fatigue** | Users faced 15+ field forms for simple operations | V4 uses wizard-style flows with auto-populated fields; 3-5 fields max per step |
| **Navigation Confusion** | Users couldn't find linked documents | V4 introduces a unified command palette + contextual navigation |
| **Data Entry Errors** | Manual entry caused cascading errors | V4 auto-fills from upstream documents; validates in real-time |
| **Training Overhead** | Each module needed separate training | V4 AI assistant guides users through any workflow via natural language |
| **Workflow Visibility** | Users lost track of where they were in a flow | V4 adds visual flow trackers showing Lead → Quote → Order → Delivery → Invoice → Payment |

### 1.3 The V4 Equation

```
V4 = V3 Architecture (Preserved)
   + UX Revolution (Guided Flows, Wizard CRUD, Auto-Population)
   + AI Layer (Natural Language → System Operations)
   + Module Completeness (All ERPNext DocTypes)
   + Multi-Tenant Deployment (Docker, VPS, Upstream Source of Truth)
   + Brand Rebirth (Obsidian ERP by VersaLabs Studio)
```

---

## 2. V3 → V4 Migration Strategy

### 2.1 What STAYS (V3 Preserved Architecture)

These V3 systems are **production-proven** and will NOT change:

| System | File(s) | Why Preserved |
|---|---|---|
| Schema-First Types | `scripts/generate-types.js`, `types/doctype-types.ts` | Zero type drift — works perfectly |
| Factory Pattern | `lib/api-factory.ts`, `hooks/generic/` | 75% boilerplate reduction — proven |
| DocType Config Registry | `lib/doctype-config.ts` | Single source of truth — essential |
| Query Key Factory | `lib/query-keys.ts` | Cache invalidation — solved problem |

| System | File(s) | Why Preserved |
|---|---|---|
| Frappe Client | `lib/frappe-client.ts` | Stable SDK wrapper |
| Zod Schemas | `lib/schemas/doctype-schemas.ts` | Runtime validation — critical |
| API Route Structure | `app/api/{module}/{doctype}/` | RESTful, clean, works |
| Theme System | `lib/theme-context.tsx`, OKLCH CSS | Premium dual-theme — keep |
| Form Components | `components/form/` | Type-safe form fields — keep |

### 2.2 What CHANGES (V4 Increments)

| Area | V3 State | V4 Target | Migration |
|---|---|---|---|
| **Page Layouts** | Standard list → detail → edit | Wizard flows, guided CRUD, dashboard-first | Rebuild page templates |
| **Navigation** | Sidebar-only | Sidebar + Command Palette + Flow Tracker + AI | Add new nav components |
| **CRUD Operations** | Manual form fills | Auto-populated wizard steps | New SmartForm engine |
| **AI Layer** | Non-existent | Full NL → Action execution | New `lib/ai/` subsystem |
| **Dashboard** | Basic module hubs | KPI-rich, action-oriented home | Rebuild dashboards |
| **Branding** | Pana Promotion / VersaForge | Obsidian ERP by VersaLabs Studio | Theme + metadata |
| **Multi-Tenant** | Single instance | Docker + tenant isolation | New infra layer |
| **Module Coverage** | ~60% ERPNext | 100% ERPNext equivalent | Add missing modules |

### 2.3 V3 Module Completion Status (Starting Point for V4)

| Module | V3 Status | V4 Action Required |
|---|---|---|
| **Stock: Item** | ✅ Complete (Golden Template) | UX overhaul only |
| **Stock: Warehouse** | ✅ Complete | UX overhaul only |
| **Stock: Material Request** | 🟡 Semi-Complete | Complete + UX overhaul |
| **Stock: Stock Entry** | 🟡 Semi-Complete | Complete + UX overhaul |
| **Stock: Delivery Note** | 📝 Docs Only | Full build + UX |

| Module | V3 Status | V4 Action Required |
|---|---|---|
| **CRM: Customer** | ✅ Complete | UX overhaul + Master Module upgrade |
| **CRM: Lead** | ✅ Complete | UX overhaul + edit propagation |
| **CRM: Contact** | ✅ Complete (Golden Template 2) | UX overhaul only |
| **CRM: Address** | ✅ Complete | UX overhaul only |
| **Sales: Quotation** | ✅ Complete | UX overhaul + attachment support |
| **Sales: Sales Order** | ✅ Complete | UX overhaul + auto→WO flow |
| **Manufacturing: BOM** | ✅ Complete | UX + Quality Inspection + Scrap/Loss |
| **Manufacturing: Work Order** | ✅ Complete | UX + UOM roundup fix |
| **Manufacturing: Workstation** | ✅ Complete | UX + flow work on Operation link |
| **Manufacturing: Operation** | ✅ Complete | UX + fixed time option |
| **Buying: Supplier** | ✅ Complete | UX overhaul only |
| **Buying: Purchase Order** | ✅ Complete | UX + approval workflow + auto-repeat |
| **Accounting: Sales Invoice** | 📝 Docs Only | Full build + UX |
| **Accounting: Purchase Invoice** | 📝 Docs Only | Full build + UX |
| **Accounting: Payment Entry** | 📝 Docs Only | Full build + UX |
| **Accounting: Journal Entry** | 📝 Docs Only | Full build + UX |
| **HR: Employee** | ✅ Complete | UX overhaul only |

### 2.4 NEW Modules for V4 (Not in V3)

| Module | DocTypes | Priority |
|---|---|---|
| **Request for Quotation** | RFQ, Supplier Quotation | High |
| **Product Bundle** | Product Bundle, Product Bundle Item | High |
| **Item Price Master** | Item Price (extended) | High |
| **Quality Inspection** | Quality Inspection, QI Template | High |
| **Project (Master)** | Project, Task, Timesheet | High |
| **Tax Master** | Tax Category, Tax Template | Medium |
| **Terms & Conditions Master** | Terms and Conditions (extended) | Medium |

| Module | DocTypes | Priority |
|---|---|---|
| **Reseller Utility** | Reseller, Commission | Medium |
| **Activity Tracking** | Activity Log, Comment | Medium |
| **Item Export** | Export Template, Export Log | Low |

---

# 3. Product Vision & Market Positioning

## 3.1 Obsidian ERP: The Pitch

> **"The ERP that runs your business while you run your business."**

Obsidian ERP is a **plug-and-play enterprise resource planning system** for Small and Medium Enterprises (SMEs) across any industry. Unlike traditional ERP systems that require months of training and dedicated IT staff, Obsidian ERP is designed so that **a shop floor operator in Addis Ababa or a procurement manager in Nairobi can use it on day one** — with zero training.

## 3.2 Three Pillars of Obsidian ERP

```
╔═══════════════════════════════════════════════════════════════════╗
║                                                                   ║
║   PILLAR 1: GUIDED SIMPLICITY                                     ║
║   ──────────────────────────                                      ║
║   Every operation is a guided flow, not a form.                   ║
║   Users click buttons and make choices. The system fills in the rest.
║   3-step wizards replace 20-field forms.                          ║
║                                                                   ║
║   PILLAR 2: AI-POWERED AUTOMATION                                 ║
║   ────────────────────────────────                                ║
║   "Create a sales order for Abebe's last quotation"               ║
║   Natural language commands execute real system operations.       ║
║   The AI understands context, fills forms, validates data,        ║
║   and asks for confirmation before executing.                     ║
║                                                                   ║
║   PILLAR 3: COMPLETE & DEPLOYABLE                                 ║
║   ──────────────────────────────                                  ║
║   Every module. Every workflow. Every report.                     ║
║   No half-built features. No "coming soon".                       ║
║   Multi-tenant Docker deployment. VPS-ready.                      ║
║   Upstream source of truth for all client instances.              ║
║                                                                   ║
╚═══════════════════════════════════════════════════════════════════╝
```

## 3.3 Target Market

| Segment | Description | Example |
|---|---|---|
| **Primary** | Manufacturing SMEs in East Africa (Ethiopia, Kenya, Uganda) | Printing shops, garment factories, food processors |
| **Secondary** | Service-based SMEs needing invoicing + CRM | Consulting firms, agencies, logistics companies |
| **Tertiary** | Reseller/White-Label Partners | IT service companies deploying ERP to their own clients |

### 3.4 Competitive Advantage

| vs Competitor | Obsidian ERP Advantage |
|---|---|
| **ERPNext (direct)** | Dramatically simpler UX, AI-powered, modern React frontend, Ethiopian localization |
| **SAP B1** | 10x cheaper, cloud-native, no consultant required |
| **Odoo** | Faster implementation, no module upselling, guided workflows |
| **QuickBooks** | Full manufacturing + inventory (QuickBooks can't do this) |
| **Spreadsheets** | Automated workflows, audit trail, multi-user, AI assistant |

### 3.5 Pilot Strategy

| Phase | Timeline | Activity |
|---|---|---|
| **Alpha (Internal)** | Month 1-2 | VersaLabs team uses Obsidian ERP for internal operations |
| **Beta (3 Clients)** | Month 3-4 | Deploy to 3 Addis Ababa SMEs via Docker on VPS |
| **GA (Public)** | Month 5+ | Open onboarding portal, reseller program, SaaS pricing |

---

## 4. Technology Stack

### 4.1 Core Stack (Preserved from V3)

| Layer | Technology | Version | Purpose |
|---|---|---|---|
| **Framework** | Next.js | 16.x (App Router) | Full-stack React framework |
| **Language** | TypeScript | 5.9+ (Strict Mode) | Type safety — non-negotiable |
| **Styling** | Tailwind CSS | v4.x | Utility-first CSS with OKLCH |
| **State** | TanStack Query | v5.x | Server state management |
| **Forms** | React Hook Form | v7.x | Performant form handling |
| **Validation** | Zod | v3.x | Runtime type validation |
| **Backend** | Frappe Framework | v15 | REST API provider |

| Layer | Technology | Version | Purpose |
|---|---|---|---|
| **Icons** | Lucide React | Latest | Consistent iconography |
| **Animations** | Framer Motion | v12.x | Micro-interactions & transitions |
| **Notifications** | Sonner | v1.x | Toast notifications |
| **Charts** | Recharts | v2.x | Dashboard visualizations |

### 4.2 NEW Dependencies for V4

| Package | Purpose | Justification |
|---|---|---|
| `openai` (OpenRouter-compatible) | AI SDK for OpenRouter API | NL → Action execution engine |
| `ai` (Vercel AI SDK) | Streaming AI responses, tool calling | Structured AI output with function calls |
| `zustand` | Lightweight global state | AI conversation state, wizard state, command palette |
| `@tanstack/react-table` | Advanced data tables | List views with inline editing, sorting, grouping |
| `react-joyride` | Onboarding tours | First-time user guided walkthroughs |
| `@dnd-kit/core` | Drag-and-drop | Kanban boards, workflow builders |

### 4.3 Infrastructure Stack (NEW for V4)

| Component | Technology | Purpose |
|---|---|---|
| **Containerization** | Docker + Docker Compose | Multi-service orchestration |
| **Reverse Proxy** | Traefik or Nginx | SSL, routing, load balancing |
| **Database** | MariaDB (Frappe default) | Per-tenant database isolation |
| **Cache** | Redis | Session store, queue, cache |
| **VPS** | Any Linux VPS (Hetzner, DigitalOcean, Vultr) | Production hosting |
| **CI/CD** | GitHub Actions | Automated build, test, deploy |
| **Monitoring** | Uptime Kuma + Grafana | Health checks, metrics |

---

## 5. Core Architecture (Preserved from V3)

> **This section is an explicit confirmation that V3's architectural DNA is carried forward unchanged.**

### 5.1 The Six Pillars (Unchanged)

| # | Pillar | V4 Status |
|---|---|---|
| P1 | Schema-First Development | ✅ Preserved — `generate-types.js` continues |
| P2 | Factory Pattern Architecture | ✅ Preserved — `api-factory.ts` + generic hooks |
| P3 | Extreme Modularization | ✅ Preserved — domain-first directory structure |
| P4 | Premium UI Standard | ✅ Preserved + Enhanced — guided flow UX added |
| P5 | Documentation as Architecture | ✅ Preserved — V4 docs replace V3 docs |
| P6 | End-to-End Type Safety | ✅ Preserved — TypeScript strict + Zod |

### 5.2 Schema-First Flow (Unchanged)

```
Frappe DocType → generate-types.js → types/doctype-types.ts + lib/schemas/
                                              ↓
                                     lib/doctype-config.ts (registry)
                                              ↓
                                     lib/query-keys.ts (cache keys)
                                              ↓
                                     lib/api-factory.ts (route handlers)
                                              ↓
                                     hooks/generic/ (CRUD hooks)
                                              ↓
                                     app/{module}/{doctype}/ (pages)
```

### 5.3 Factory Pattern (Unchanged)

```
// API Routes — SAME AS V3
export const GET = createListHandler("Item", { allowedFields: [...] });
export const POST = createCreateHandler("Item");

// Hooks — SAME AS V3
const { data } = useFrappeList<Item>("Item", { filters: [...] });
const create = useFrappeCreate<Item>("Item");
```

### 5.4 Directory Structure (Extended for V4)

```
obsidian-erp/
├── app/
│   ├── api/                          # API Routes (V3 — preserved)
│   │   └── {module}/{doctype}/
│   │       ├── route.ts              # GET list, POST create
│   │       └── [name]/route.ts       # GET, PUT, DELETE
│   │
│   ├── api/ai/                       # NEW: AI API routes
│   │   ├── chat/route.ts             # Chat completions
```

```
│   │   ├── execute/route.ts          # Action execution
│   │   └── context/route.ts          # Context resolution
│   │
│   ├── {module}/                     # Page Routes (V3 pattern, V4 UX)
│   │   └── {doctype}/
│   │       ├── page.tsx              # List page (V4: enhanced)
│   │       ├── new/page.tsx          # Create page (V4: wizard flow)
│   │       └── [name]/
│   │           ├── page.tsx          # Detail page (V4: action-oriented)
│   │           └── edit/page.tsx     # Edit page (V4: wizard flow)
│   │
│   └── onboarding/                   # NEW: Tenant onboarding
│       └── page.tsx
│
├── components/
│   ├── ui/                           # Primitive UI (V3 — preserved)
│   ├── smart/                        # Smart Components (V3 + V4 additions)
│   ├── form/                         # Form Components (V3 — preserved)
│   ├── Layout/                       # Layout Components (V3 + V4 mods)
│   ├── flows/                        # NEW: Guided Flow components
│   │   ├── FlowWizard.tsx            # Multi-step wizard engine
│   │   ├── FlowTracker.tsx           # Visual workflow progress
│   │   ├── FlowStep.tsx              # Individual wizard step
│   │   └── FlowAutoFill.tsx          # Auto-population engine
│   ├── ai/                           # NEW: AI interface components
│   │   ├── AICopilot.tsx             # Main AI panel
│   │   ├── AIMessage.tsx             # Chat message component
│   │   ├── AIActionCard.tsx          # Action confirmation card
│   │   └── AIContextBadge.tsx        # Current context display
│   ├── command/                      # NEW: Command palette
│   │   └── CommandPalette.tsx        # Global search + actions
│   └── dashboard/                    # NEW: Module dashboards
│       ├── KPICard.tsx               # Key metric display
│       ├── ActionCard.tsx            # Quick action button
│       └── FlowStatus.tsx            # Workflow status summary
│
├── hooks/
│   ├── generic/                      # Generic Frappe Hooks (V3 — preserved)
│   ├── useDeleteWithConfirmation.ts  # (V3 — preserved)
│   ├── useExport.ts                  # (V3 — preserved)
│   ├── useWizardFlow.ts              # NEW: Wizard state management
│   ├── useAutoFill.ts                # NEW: Auto-population logic
│   ├── useCommandPalette.ts          # NEW: Command palette state
│   └── useAI.ts                      # NEW: AI interaction hook
│
├── lib/
│   ├── doctype-config.ts             # DocType Registry (V3 — preserved)
│   ├── query-keys.ts                 # Query Key Factory (V3 — preserved)
│   ├── api-factory.ts                # API Route Factories (V3 — preserved)
│   ├── frappe-client.ts              # Frappe SDK Wrapper (V3 — preserved)
│   ├── theme-context.tsx             # Theme Provider (V3 — preserved)
│   ├── utils.ts                      # Utilities (V3 — preserved)
│   ├── schemas/                      # Zod Schemas (V3 — preserved)
│   ├── ai/                           # NEW: AI subsystem
```

```
│   │   ├── ai-client.ts            # OpenRouter client with fallback
│   │   ├── ai-tools.ts             # Function definitions for tool calling
│   │   ├── ai-context.ts           # Context resolution engine
│   │   ├── ai-executor.ts          # Action execution engine
│   │   └── ai-config.ts            # Model configuration + routing
│   ├── flows/                      # NEW: Flow engine
│   │   ├── flow-definitions.ts     # Workflow step definitions
│   │   ├── flow-auto-fill.ts       # Auto-population rules
│   │   └── flow-validation.ts      # Step-level validation
│   └── tenant/                     # NEW: Multi-tenant utilities
│       ├── tenant-config.ts        # Tenant resolution
│       ├── tenant-middleware.ts    # Request-level tenant routing
│       └── tenant-branding.ts      # Per-tenant branding
│
├── types/
│   ├── doctype-types.ts            # Generated Types (V3 — preserved)
│   ├── ai-types.ts                 # NEW: AI system types
│   ├── flow-types.ts               # NEW: Flow engine types
│   └── tenant-types.ts            # NEW: Multi-tenant types
│
├── docs/
│   ├── v3/                         # V3 docs (archived, read-only)
│   └── v4/                         # V4 docs (this document set)
│       ├── ARCHITECTURE_V4_PART1_FOUNDATION.md      # This file
│       ├── ARCHITECTURE_V4_PART2_UX_REVOLUTION.md   # UX patterns
│       ├── ARCHITECTURE_V4_PART3_AI_INTEGRATION.md  # AI system
│       ├── ARCHITECTURE_V4_PART4_DEPLOYMENT.md      # Multi-tenant + ops
│       ├── BUSINESS_WORKFLOW_PART1_LEAD_TO_CASH.md  # Business flow
│       ├── BUSINESS_WORKFLOW_PART2_MODULE_SPECS.md  # Module specs
│       └── BUSINESS_WORKFLOW_PART3_AUTOMATION.md    # Automation rules
│
├── docker/                         # NEW: Docker configuration
│   ├── Dockerfile                  # Next.js production build
│   ├── docker-compose.yml          # Full stack orchestration
│   ├── docker-compose.dev.yml      # Development environment
│   └── nginx.conf                  # Reverse proxy config
│
└── scripts/
    ├── generate-types.js           # Type Generation (V3 — preserved)
    ├── setup-tenant.sh             # NEW: Tenant provisioning
    └── seed-data.js                # NEW: Demo data seeder
```

---

# 6. V4 Architectural Increments

## 6.1 Increment 1: SmartForm Engine (Guided CRUD)

The biggest UX change in V4. Instead of presenting a blank form with 15+ fields, V4 uses a **wizard-based SmartForm** that:

1. **Auto-fills** from upstream documents (e.g., Sales Order auto-fills from Quotation)
2. **Groups fields** into logical steps (3-5 fields max per step)

3. **Validates per step** before advancing
4. **Shows context** — what document this creates, what it links to
5. **Confirms before saving** — preview of what will be created

```
// V4 SmartForm Engine — conceptual API
interface FlowDefinition {
  doctype: string;
  source?: {                          // Auto-fill source
    doctype: string;
    fetchFields: string[];            // Fields to copy from source
  };
  steps: FlowStep[];
  confirmation: {
    title: string;
    summaryFields: string[];          // Fields to show in preview
  };
}

interface FlowStep {
  id: string;
  title: string;
  description: string;
  fields: FlowField[];
  autoFillRules?: AutoFillRule[];     // Rules to auto-populate
  validationSchema: z.ZodSchema;      // Per-step Zod validation
}

interface FlowField {
  name: string;
  label: string;
  type: 'input' | 'select' | 'frappe-select' | 'date' | 'number' | 'switch';
  required: boolean;
  autoFilled?: boolean;               // If true, field is read-only (auto-
populated)
  helpText?: string;
}
```

### 6.2 Increment 2: Command Palette

Global keyboard-activated (Cmd/Ctrl+K) search and action palette:

```
interface CommandPaletteAction {
  id: string;
  label: string;
  icon: LucideIcon;
  category: 'navigate' | 'create' | 'search' | 'ai';
  shortcut?: string;
  action: () => void;
}
```

```
// Examples:
// "Go to Sales Orders" → navigates to /sales/sales-order
// "Create new Customer" → opens wizard at /crm/customer/new
// "Find ITEM-001" → searches across all DocTypes
// "Ask AI: What's our best selling item?" → opens AI panel
```

### 6.3 Increment 3: Flow Tracker

Visual indicator showing the user's position in the Lead → Cash flow:

```
Lead ──→ Opportunity ──→ Quotation ──→ Sales Order ──→ Work Order ──→ Delivery ──→
Invoice ──→ Payment
  ●          ●             ●             ◉             ○             ○
○          ○
                                      (current)
```

This component appears on every transactional document page, showing:

- Where the document sits in the business flow
- What came before (clickable links)
- What comes next (action buttons)
- Current status of each stage

### 6.4 Increment 4: Enhanced DocType Config

V4 extends the centralized config with flow and AI metadata:

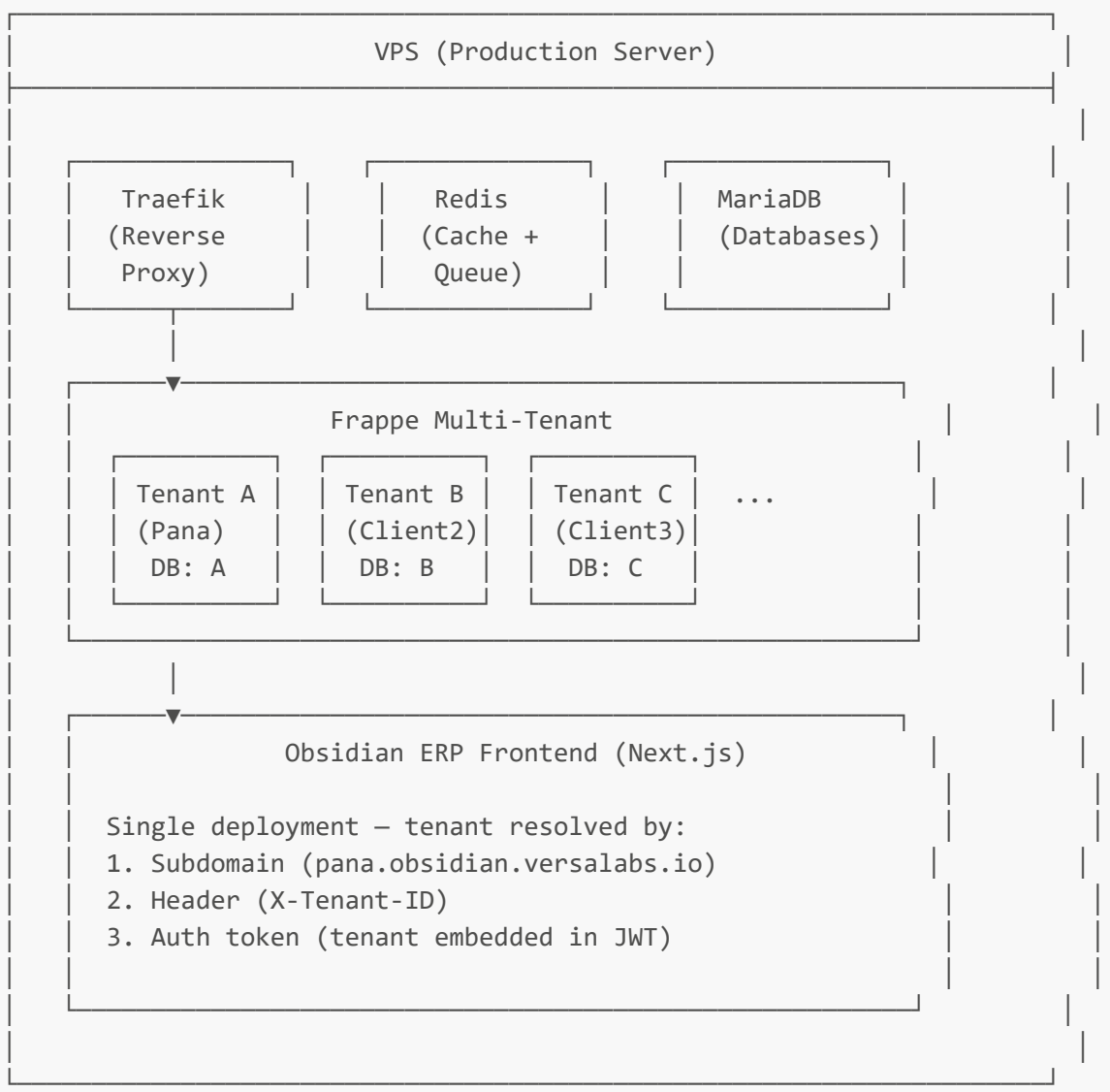
```
// V4 Extension to DocTypeConfig
export interface DocTypeConfigV4 extends DocTypeConfig {
  // V4: Flow Engine metadata
  flow?: {
    position: number;                // Position in lead-to-cash flow (1-8)
    upstreamDoctype?: string;        // What feeds into this
    downstreamDoctype?: string;      // What this creates
    autoFillFrom?: string[];         // DocTypes to auto-fill from
  };

  // V4: AI metadata
  ai?: {
    createVerb: string;              // "create", "place", "submit"
    naturalName: string;             // "sales order", "quotation"
    searchableFields: string[];      // Fields AI can query by
    actionable: boolean;             // Can AI create/modify this?
  };

  // V4: UX metadata
  ux?: {
    wizardSteps?: number;            // Number of wizard steps
    dashboardWidget?: 'counter' | 'chart' | 'list' | 'kanban';
```

```
    requiresApproval?: boolean;       // Needs approval workflow?
    hasAttachments?: boolean;         // Supports file attachments?
  };
}
```

---

## 7. Multi-Tenant Architecture

### 7.1 Deployment Model

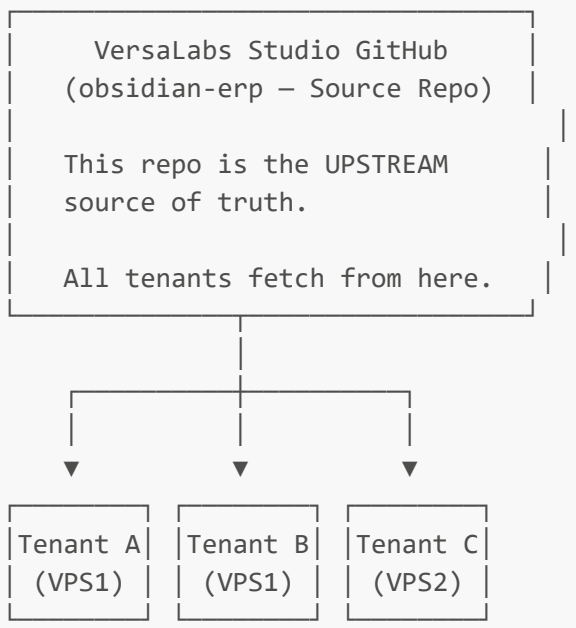



### 7.2 Tenant Isolation Strategy

| Layer | Isolation | Implementation |
|---|---|---|
| **Database** | Full isolation (separate DB per tenant) | Frappe multi-tenancy (bench) |
| **Frontend** | Shared code, tenant-resolved config | Middleware resolves tenant from subdomain |
| **API** | Routed to correct Frappe site | `X-Frappe-Site-Name` header |

| Layer | Isolation | Implementation |
|---|---|---|
| **Branding** | Per-tenant logo, colors, company name | `tenant-branding.ts` config |
| **Storage** | Isolated file storage per tenant | Frappe `sites/{site}/` structure |

### 7.3 Source of Truth Architecture




```
Deploy flow:
1. git push to main (source of truth)
2. GitHub Actions builds Docker image
3. Docker image pushed to registry
4. VPS pulls latest image
5. docker compose up (zero-downtime)
```

---

## 8. Brand Identity: Obsidian ERP

### 8.1 Brand Assets

| Asset | Old (V3) | New (V4) |
|---|---|---|
| **Product Name** | Pana Promotion ERP / VersaForge ERP | **Obsidian ERP** |
| **Company** | — | **VersaLabs Studio** |
| **Logo** | Pana logo | New logo (provided by Kidus) |
| **Tagline** | — | "The ERP that runs your business while you run your business." |
| **Domain** | — | `obsidian.versalabs.io` (proposed) |

### 8.2 Theme Tokens (V4 Rebrand)

The OKLCH color system is preserved but rebranded:

| Token | Purpose | Light Mode | Dark Mode |
|---|---|---|---|
| `--color-obsidian-brand` | Primary brand color | `oklch(0.35 0.02 260)` | `oklch(0.55 0.18 265)` |
| `--color-obsidian-accent` | Accent/CTA color | `oklch(0.65 0.20 165)` | `oklch(0.70 0.20 165)` |
| `--color-background` | Page background | `oklch(0.98 0.005 260)` | `oklch(0.13 0.015 260)` |
| `--color-card` | Card surfaces | `oklch(1 0 0)` | `oklch(0.17 0.015 260)` |
| `--color-foreground` | Primary text | `oklch(0.15 0.02 260)` | `oklch(0.95 0.005 260)` |

### 8.3 Code-Level Brand Changes

```
// app/layout.tsx — V4 metadata
export const metadata: Metadata = {
  title: "Obsidian ERP",
  description: "Enterprise Resource Planning by VersaLabs Studio — Guided Simplicity for SMEs",
};

// localStorage key migration
const THEME_KEY = 'obsidian-erp-theme';  // was: 'pana-erp-theme'
```

---

## Summary

Part 1 establishes:

1. ✅ **Strategic context** — why V4, what V3 taught us
2. ✅ **Migration strategy** — what stays, what changes, module status
3. ✅ **Product vision** — Obsidian ERP positioning, market, pilot plan
4. ✅ **Technology stack** — V3 preserved + V4 additions
5. ✅ **Core architecture** — Six Pillars confirmed, schema-first preserved
6. ✅ **Architectural increments** — SmartForm, Command Palette, Flow Tracker
7. ✅ **Multi-tenant design** — Docker, VPS, source of truth model
8. ✅ **Brand identity** — Obsidian ERP by VersaLabs Studio

**Next:** Part 2 covers the full UX Revolution — page templates, wizard flows, dashboard design, and the new Golden Template for V4.

---

*© 2026 VersaLabs Studio. All rights reserved.*